

BACKEND REQUIREMENTS SPECIFICATION — DEV HUB

Document Version: 1.0

Date: 2026-05-02

Status: Greenfield Development

Purpose: Provide the complete backend blueprint to support the already-defined Dev Hub UI. The backend must be built from scratch with no existing database or API.

1. PROJECT OVERVIEW

Dev Hub is a developer context engine that captures raw thoughts, links them to code changes, and helps developers seamlessly resume work on projects. The user interface has already been designed (see screenshots and prototype). The backend is entirely new and must:

- Support multi-provider authentication (Google, GitHub, email) via Supabase Auth.
- Provide project management with frozen state snapshots for context retention.
- Enable time-boxed work sessions with an intention declaration.
- Power a real-time “Weird Stuff Feed” for logging developer thoughts (with type categorization).
- Serve activity views: commits with AI summaries, a file heatmap, and linked thoughts.
- Host a Feature Brainstormer for capturing, validating, and promoting feature ideas.
- Deliver quick-start suggestions and GitHub issue integration (Phase 2).
- Conduct end-of-session audits and track unresolved threads.
- Real-time updates via WebSockets.

All specifications must be fully implementable by a backend developer without requiring additional UI clarifications.

2. SYSTEM ARCHITECTURE

- **Framework:** FastAPI (Python) — async, WebSocket support, auto-generated docs.
- **Database:** PostgreSQL (via Supabase) + pgvector for embedding storage and vector similarity search.

- **Real-Time:** WebSocket connections managed by a connection manager (broadcast per session).
- **Background Jobs:** Celery / APScheduler for heavy AI processing (embedding generation, commit-log linking, session freezing).
- **External Services:**
 - **Supabase Auth** — user management, social OAuth, JWT issuing.
 - **Gemini Pro API** — embeddings, code-diff analysis, summarisation, suggestions.
 - **GitHub API** — OAuth token exchange, repository events, issue fetching, diffs.

Frontend Communication: REST over HTTPS + WebSocket (wss://).

3. DATABASE SCHEMA (GREENFIELD)

The following schema is the **complete initial definition**. All tables, indexes, and constraints must be created by the backend developer.

3.1 Table Definitions

sql

```
-- Enable necessary extensions
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "vector";

-----

-- developers (auth handled by Supabase, this table mirrors the Supabase user)
-----

CREATE TABLE developers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  supabase_user_id UUID UNIQUE NOT NULL, -- from auth.users
  email TEXT,
  username TEXT,
  github_id TEXT,
  github_token_encrypted TEXT, -- encrypted GitHub access token
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

3.2 Entity Relationship Overview

text

```
developers —< projects —< sessions —< logs
projects —< feature_ideas —< sessions (promoted)
logs —< feature_ideas (optional)
projects —< file_activity
projects —< unresolved_threads
projects —< project_issues_cache (optional)
commits —
logs ————— many-to-many via linked IDs (denormalized)
```

4. AUTHENTICATION & AUTHORISATION

The frontend expects **Google, GitHub, and Email** login flows. All authentication is delegated to Supabase Auth.

Backend Implementation:

- Configure Supabase project with Google, GitHub, and Email providers.
- No custom user passwords stored; rely on Supabase's `auth.users`
- On successful sign-in, Supabase returns a JWT to the frontend.
- Every protected endpoint must validate the JWT using Supabase's public key (via `supabase-py` helper or manual JWKS check). Extract `sub` (user ID) to identify the developer.

User-developer mapping:

- When a user first accesses the backend, check if a `developers` row with `supabase_user_id = sub` exists; if not, create one and store any GitHub-specific metadata (e.g., `github_id`, `email`).
- All subsequent requests use the `developer.id` (UUID) for ownership checks.

Authorization Middleware:

- All endpoints (except webhooks) require a valid JWT.
- Ownership: Before modifying a resource (project, session, log), verify `developer_id` matches the authenticated user.

5. API ENDPOINT SPECIFICATIONS

Base URL: `/api/v1` (versioned). All endpoints return JSON.

5.1 Project & Dashboard

Method	Path	Description
GET	/projects	List user's projects (with unresolved count)
GET	/projects/{id}	Get project details + frozen state
POST	/projects	Create a new project (name, optional repo)
GET	/projects/{id}/suggestions	AI/rule-based suggestion for next work
GET	/projects/{id}/unresolved-count	Return { count: n } for badge

GET	/projects/{id}/github/issues	(Phase 2) Cached GitHub issues
-----	------------------------------	---

Example: GET /projects/{id}

json

```
{
  "id": "uuid",
  "name": "NodeLink App",
  "github_repo_full_name": "alice/radarapp",
  "frozen_state": {
    "briefing": "Welcome back. Last time you left the RSSI math half-baked...",
    "hot_files": [
      { "path": "RadarScreen_2.kt", "heat_level": "hot", "modification_count": 23 },
      { "path": "NodeMeshService_2.kt", "heat_level": "stable",
    "modification_count": 2 }
    ],
    "unresolved_threads": [
      { "id": "uuid", "content": "GATT connection returning null on Sony Xperia" }
    ],
    "session_stats": { "total_sessions": 5, "total_logs": 23 }
  },
  "unresolved_count": 2
}
```

Example: GET /projects/{id}/suggestions

json

```
{
  "suggestion": "Based on your recent activity, you might want to continue working on the BLE service coroutine refactor. Last session, you left a question about this unresolved."
}
```

5.2 Session Management

Method	Path	Description
POST	/sessions	Start a new session
GET	/sessions/{id}	Get session details
POST	/sessions/{id}/end	End session (with audit)
POST	/sessions/{id}/audit	Mark specific unresolved logs as threads

POST /sessions **request:**

json

```
{
  "project_id": "proj-uuid",
  "declared_intent": "Fixing: GATT connection null on Sony Xperia"
}
```

Response 201:

json

```
{
  "id": "session-uuid",
  "declared_intent": "Fixing: GATT connection null on Sony Xperia",
  "status": "active",
}
```

```
"started_at": "2026-05-02T14:00:00Z"
}
```

POST /sessions/{id}/end **response:**

json

```
{
  "status": "ended",
  "audit": {
    "logs_count": 5,
    "commits_pushed": 2,
    "unresolved": [
      {
        "id": "log-uuid",
        "content": "Should I refactor the BLE service to use coroutines?"
      }
    ]
  }
}
```

5.3 Logging (Weird Stuff Feed)

Method	Path	Description
POST	/logs	Create a new log entry
GET	/sessions/{id}/logs?type=bug	List logs for a session (optional type filter)

POST /logs **request:**

json

```
{
  "session_id": "session-uuid",
  "content": "The GATT connection is returning null on the Sony Xperia.",
  "log_type": "bug"
}
```

Response 201:

json

```
{
  "id": "log-uuid",
  "content": "The GATT connection is returning null on the Sony Xperia.",
  "log_type": "bug",
  "is_resolved": false,
  "linked_commit_ids": [],
  "created_at": "2026-05-02T14:05:00Z"
}
```

5.4 Activity / Commits

Method	Path	Description
GET	/commits	List commits for a project (?project_id=...)
GET	/sessions/{id}/heatmap	Get file heatmap for a session

GET /commits?project_id=... response:

json

```
{
  "commits": [
    {
      "id": "commit-uuid",
      "github_commit_hash": "def456",
      "message": "fix: clamp RSSI values to prevent NaN",
      "ai_summary": "You fixed the distance calculation NaN issue by clamping RSSI values below -100.",
      "changed_files": ["SignalMath.kt", "BleViewModel.kt"],
      "linked_log_ids": ["log-uuid-1"],
      "pushed_at": "2026-05-02T13:30:00Z"
    }
  ]
}
```

5.5 Feature Brainstormer

Method	Path	Description
POST	/feature-ideas	Create a new feature idea
GET	/feature-ideas? project_id=.. &status=.. .	List ideas
POST	/feature-ideas/{id}/validate	Trigger validation (manual or AI)

POST	/feature-ideas/{id}/promote	Promote idea to active session
------	-----------------------------	--------------------------------

POST /feature-ideas **request:**

json

```
{
  "project_id": "proj-uuid",
  "description": "Add real-time device tracking with live position updates on the radar",
  "log_id": "optional-log-uuid"
}
```

Response 201: { "id": "idea-uuid", "status": "draft", ... }

POST /feature-ideas/{id}/validate **(manual override)**

Request: { "affected_files": ["RadarScreen.kt", "DeviceTracker.kt"] }

Response:

json

```
{
  "status": "validated",
  "validated_at": "...",
  "affected_files": ["RadarScreen.kt", "DeviceTracker.kt"]
}
```

If no files provided, the backend triggers a background AI job to determine affected files (Phase 2).

POST /feature-ideas/{id}/promote :

- Ends any currently active session for the project.
- Creates a new session with `declared_intent = feature_idea.description`.
- Updates `feature_ideas.status = 'promoted'` `promoted_session_id = new_session.id`.
- Returns: { "status": "promoted", "session": { ... } }.

5.6 Webhooks (Phase 2 Only)

POST /webhooks/github

- Validates HMAC SHA-256 signature.
- Processes push event: inserts commits, triggers background job for AI linking and heatmap update.
- Returns 200 immediately (processing is async).

6. REAL-TIME WEBSOCKET SPECIFICATION

Connection: ws://<host>/ws/session/{session_id}?token=<JWT>

- Backend validates JWT on upgrade; only the session owner and authorized collaborators (future) can connect.

Server → Client Messages:

Event Type	Payload Example	Trigger
log_created	<pre>{ "type": "log_created", "data": { ... log object } }</pre>	After log insertion
commit_processed	<pre>{ "type": "commit_processed", "data": { "commit": ..., "heatmap_update": ... } }</pre>	After webhook commit processing
intent_resolved	<pre>{ "type": "intent_resolved", "intent_resolved": ... }</pre>	When AI updates

	lved", "data": { "percentage" : 75 } }	resolution
file_heatmap_updated	{ "type": "file_heatmap_updated", "data": [...] }	After heatmap recalc
feature_id_created	{ "type": "feature_id_created", "data": { ... } }	When a new idea is created
feature_id_validated	{ "type": "feature_id_validated", "data": { ... } }	After validation completes
session_ended	{ "type": "session_ended" }	When session is ended by any client

Client → Server Messages:

- **heartbeat** (every 30s) – to keep connection alive.
- **subscribe** – optional, for future channel filtering.

Connection Manager:

A single `SessionConnectionManager` maintains a mapping of `session_id` -> list of

WebSocket connections. On disconnect, clean up the list. Broadcasts are send-only, catching and ignoring connection errors.

7. BACKGROUND JOBS & AI INTEGRATIONS

All tasks that involve external API calls, heavy computation, or long-running work must be run asynchronously via a task queue (e.g., Celery with Redis, or APScheduler for simpler setups).

Job 1: Embed Log

- Trigger: after a new log is created.
- Action: Call Gemini embedding API (`models/embedding-001`) with log content, dimension 768.
- Update: `logs.embedding`.
- Note: This is needed for future semantic search; can be deferred until Phase 2.

Job 2: Process GitHub Push

- Trigger: after `POST /webhooks/github` validates event.
- For each commit:
 - A. Fetch full diff from GitHub API.
 - B. INSERT commit record.
 - C. Call Gemini to summarise diff → `ai_summary`.
 - D. Call Gemini to match diff with recent unresolved logs →
update `logs.linked_commit_ids` `logs.is_resolved`.
 - E. Update `file_activity` (increment `modification_count`, recalc heat).
 - F. Recalculate session intent resolution % (if any).
 - G. Broadcast `commit_processed` via WebSocket.

Job 3: Freeze Project

- Trigger: after session ends (if `POST /sessions/{id}/audit` marks unresolved logs).
- Action:
 - A. Collect unresolved logs for the project.
 - B. Call Gemini to generate a 3-sentence briefing based on logs + heatmap.

C. Update `projects.frozen_state`.

D. Optional: store unresolved threads as needed.

Job 4: Validate Feature Idea (AI)

- Trigger: `POST /feature-ideas/{id}/validate` without manual files.
- Action:
 - A. Fetch repository file structure via GitHub API.
 - B. Use Gemini to predict affected files based on idea description.
 - C. Update `feature_ideas.affected_files` and status.
 - D. Broadcast update via WebSocket.

Job 5: Refresh GitHub Issues Cache (Phase 2)

- Periodic or on-demand; fetches open issues from GitHub, stores in `project_issues_cache`.

8. INTEGRATION POINTS & EXTERNAL APIS

- **Supabase Auth:** Use `supabase-py`. On each request, validate token via `supabase.auth.get_user(token)`.
- **Gemini API:**
 - Endpoints: <https://generativelanguage.googleapis.com/v1beta/models/gemini-pro:generateContent> (summaries), `models/embedding-001:embedText` (embeddings).
 - Handle `429` with exponential backoff.
- **GitHub API:**
 - OAuth: exchange code for token at https://github.com/login/oauth/access_token.
 - REST: fetch commit details, diff, repo tree, issues.
 - Webhooks: validate signature using `X-Hub-Signature-256`.
- **Encryption:** GitHub tokens stored in DB must be encrypted (e.g., using Supabase Vault or a symmetric key stored in env vars).

9. KEY DATA FLOW SCENARIOS

9.1 Returning User Opens Project

1. `GET /projects/{id}` → receives briefing, heatmap, unresolved count.
2. `GET /projects/{id}/suggestions` → AI suggestion.

3. UI displays Welcome screen.

9.2 Start Session & Log

1. `POST /sessions` with intent.
2. User types a thought → `POST /logs` with optional `log_type`.
3. Backend stores log, enqueues embedding job, broadcasts via WebSocket.
4. UI updates feed in real time.

9.3 Feature Brainstormer

1. User enters idea → `POST /feature-ideas`.
2. (Option) "Analyze with AI" → `POST /.../validate` → background job.
3. On completion, UI shows affected files.
4. "Promote" → creates a new session, user redirected to dashboard.

9.4 End Session

1. `POST /sessions/{id}/end` → returns unresolved logs.
2. Modal appears; user clicks "Log & Exit" → `POST /sessions/{id}/audit`.
3. Background job freezes project state.

10. POTENTIAL CHALLENGES & MITIGATIONS

- **Performance:** Large number of logs per session; use pagination on `GET /logs` (cursor-based).
- **AI Cost & Latency:** All AI calls are backgrounded; use placeholder summaries in MVP if needed, then introduce real AI in Phase 2.
- **WebSocket Scale:** Use Redis pub-sub if deploying multiple backend instances.
- **GitHub Rate Limits:** Cache aggressively; respect conditional requests; design webhook processing to be idempotent (deduplicate commits by hash).
- **Unresolved Count:** Optimize with a partial index on `unresolved_threads` or a denormalized counter on `projects`.
- **Feature Idea Validation Accuracy:** Allow manual file entry as a fallback; store validation method.

11. IMPLEMENTATION PHASES (RECOMMENDED)

Phase 1 – Core MVP (4 weeks)

- Database creation (all tables from Section 3).
- Auth integration (Google, GitHub, email via Supabase).
- Session lifecycle: create, log (with `log_type`), end, audit.
- Log filtering by type.
- Basic `GET /suggestions` (rule-based: if unresolved threads exist, use the most recent one).
- Feature ideas: CRUD, manual validation, promotion to session.
- Real-time WebSocket for logs and session events.
- Project list with unresolved badge.
- No GitHub or AI integrations; use mocks where required.

Phase 2 – GitHub & AI (4 weeks)

- GitHub OAuth token storage (encrypted).
- Webhook endpoint + background commit processing + commit-log linking (Gemini).
- File heatmap automated updates from commits.
- GitHub issue fetching and caching.
- AI-powered feature idea validation (real codebase analysis).
- Embedding generation for logs (for future search).

Phase 3 – Intelligence & Team (4 weeks)

- Semantic log search (via pgvector).
- Enhanced AI suggestions (using Gemini based on full project state).
- Team features and multi-user access.

12. APPENDIX: UI FEATURE REFERENCE

The backend must fully enable the following UI screens:

- **Auth Screen:** Social login buttons, email form.
- **Welcome / Dashboard (no active session):** Project selector, briefing, heatmap, recent commits, quick-start buttons.
- **Active Session:** Intent display, Pomodoro timer, Weird Stuff Feed with type badges, commit sidebar.

- **Activity Tabs:** Commits with AI summaries and linked thoughts, file changes timeline with heat indicators.
- **Feature Brainstormer:** Input form, Wire Map of validated ideas, “Promote” button.
- **Modal (End Session):** Display unresolved items, options to commit now or log as unresolved.

All data shown in screenshots must be obtainable from the API endpoints defined in this document.

This specification is the single source of truth for backend development. Any deviations should be discussed and documented herein.